



Roteiro de Aula — Laços de Repetição em Python

Prof. Bruno Melo · Universidade Positivo — Processo Seletivo Docente

Guia de condução da aula (~20 min) · `for` e `while`

Sumário

1	Abertura / gancho — 00:00 a 02:00 (slides 1 a 4)	2
2	Lógica antes da sintaxe — 02:00 a 05:00 (slides 5 a 7)	2
3	<code>for</code> passo a passo (build interativo) — 05:00 a 09:00 (slides 8 a 14)	2
4	QUADRO — 09:00 a 12:00 (ponto de pausa planejado)	3
5	<code>while</code> e <code>while True + break</code> — 12:00 a 14:00 (slides 16 a 18)	3
6	VS Code ao vivo — 14:00 a 18:00	4
7	Fechamento — 18:00 a 20:00 (slides 20 a 22)	4
8	Planos de contingência	5

9 Referências

5

Fio condutor único: um simulador de poupança — *quanto tempo até juntar dinheiro para um notebook?* — atravessa todas as etapas (pseudocódigo, slide, quadro e VS Code), dando coesão narrativa à aula.

Como ler este roteiro. Os marcadores em caixa indicam os dois momentos em que você **sai dos slides**: a caixa escura sinaliza **[IR AO QUADRO]** e a caixa azul, **[ABRIR O VS CODE]**. As falas sugeridas estão em caixas cinza.

1 Abertura / gancho — 00:00 a 02:00 (slides 1 a 4)

Objetivo: criar a necessidade do conceito com uma situação real (MASETTO, 2012).

- **00:00** (capa) — “Bom dia à banca. Sou Bruno Melo. Nos próximos 20 minutos vamos ver como fazer o computador repetir tarefas por nós: os **laços de repetição** em Python.”
- **00:20** (roteiro) — Mapa da aula em uma frase: “Começo pela lógica, passo pela sintaxe, vou ao **quadro** e termino com **código rodando ao vivo**.”
- **00:45** (gancho) — Pergunta à banca, com pausa de 2–3 s: “*Quantas vezes você já fez a mesma conta, repetindo à mão?*”
- **01:00** (por que repetir) — Avançar os fragmentos narrando: juntar dinheiro para um notebook; guardar todo mês; conferir o total; “à mão isso cansa”.

Fala-âncora: “A estrutura que expressa *repita* chama-se laço de repetição. É disso que a aula trata: repetir com controle.”

2 Lógica antes da sintaxe — 02:00 a 05:00 (slides 5 a 7)

Objetivo: desenhar o algoritmo em pseudocódigo e plantar a distinção contada x condicionada (ASCENCIO; CAMPOS, 2012).

- **02:00** (transição) — “Primeiro pensamos o problema; depois traduzimos para Python.”
- **02:15** (pseudocódigo) — Ler o pseudocódigo apontando cada linha: `total <- 0; ENQUANTO total < meta; acumula; conta meses.` **Frase-chave:** “não sabemos quantos meses serão — isso depende de uma condição. É a repetição **condicionada**.”
- **03:40** (duas perguntas) — Tabela: “Quantos meses até a meta?” leva a `while`; “Quanto em 12 meses fixos?” leva a `for`. Fechar: “mesmo problema, muda só quem decide a parada. Vamos começar pela versão contada, no `for`.”

3 for passo a passo (build iterativo) — 05:00 a 09:00 (slides 8 a 14)

Objetivo: mostrar que, em Python, o `for` **percorre um intervalo**; destacar em sequência o intervalo, a variável de controle e o corpo. Cada avanço de slide **acende** uma parte do código — aponte a linha destacada.

- **05:15** (intuição — 12 círculos) — “`range(1, 13)` gera **12 valores**; o `for` visita cada mês, um a um. Repare: o **13 fica de fora**.”
- **06:00** (visão geral) — “O `for` percorre um intervalo: a cada volta, `mes` recebe o próximo valor; o corpo é **indentado**.”
- **06:40** (o intervalo) — “`range(1, 13)` define **quantas voltas** o laço dará.”
- **07:15** (a variável de controle) — “`mes` recebe o **próximo valor** a cada volta. O avanço é **automático**: em Python não existe `mes++`.”
- **07:50** (o corpo) — “O bloco **indentado**: acumula a economia e mostra o total. A indentação diz o que pertence ao laço.”
- **08:20** (ordem de execução) — Recitar: pega o próximo valor do `range`, guarda em `mes`, executa o corpo, repete; quando o intervalo acaba, termina. “É exatamente o que vou desenhar agora.”

4 QUADRO — 09:00 a 12:00 (ponto de pausa planejado)

[IR AO QUADRO — 3 min]. Deixe o slide 15 ("AO QUADRO") projetado: ele sinaliza à banca que a ida ao quadro é intencional. Pegue o giz. O passo a passo completo está no documento `instrucoes_quadro` (PDF). Resumo: desenhe lado a lado os fluxos de `for` e `while`, com uma nota sobre `while True + break`, destacando *onde a condição é testada*.

- **09:00** Desenhar o `while`: início; `losango total < meta?`; saída **Verdadeiro** para o retângulo do corpo; **seta de retorno** ao losango; saída **Falso** para “fim”.
- **10:00** Ao lado, o `for`: mostrar que ele **percorre um intervalo** (`range`), com o avanço embutido; o número de voltas é conhecido. “Mesmo destino, caminho diferente.”
- **11:00** Nota `while True + break`: o **corpo antes, o teste (com break) no fim**; a seta de início cai direto no corpo. “Por isso roda **pelo menos uma vez** — é o substituto do do-while em Python.”
- **11:40** Circular a diferença: no `while` testa-se **em cima**; no `while True + break`, **embaixo**.

RETOMADA (11:50): guarde o giz, vire-se para a banca e diga: “Agora que vimos o fluxo no quadro, vamos ver essas formas escritas em Python.” **Avance o slide** (15 para 16, bloco `while`).

5 `while` e `while True + break` — 12:00 a 14:00 (slides 16 a 18)

Objetivo: reforço rápido, amarrando com o desenho do quadro.

- **12:00** (`while`) — Voltar ao problema da **meta**. “Condição **no topo** — igual ao losango de cima que desenhei. Se o total já bateu a meta, o corpo não roda” (DOWNEY, 2016).
- **12:50** (`while True + break`) — “Python **não tem do-while**. Para rodar **ao menos uma vez**, uso `while True`: e coloco a saída (**break**) **no fim** do corpo. Pense num menu que precisa aparecer antes de perguntar ‘deseja continuar?’”
- **13:20** (os dois lado a lado) — Tabela-resumo: contada (`for`) x condicionada (`while`); quem pode rodar 0 vez; o padrão `while True + break` para “ao menos 1 vez”.

6 VS Code ao vivo — 14:00 a 18:00

[**ABRIR O VS CODE — 4 min**]. Alt+Tab para o VS Code, já com as três abas (`exemplo_for.py`, `exemplo_while.py`, `exemplo_erro_proposital.py`) e o **terminal integrado** abertos na pasta `codigos_py`, com a fonte ampliada. O slide 19 ("VS Code ao vivo") fica projetado.

Executar (terminal integrado, com Python no PATH):

```
python exemplo_for.py
python exemplo_while.py
python exemplo_erro_proposital.py
```

Sem terminal? Use o botão **Run Python File** da extensão Python (**Ctrl+F5**).

- **14:00 — Exemplo 1 (`exemplo_for.py`):** rodar; digitar 300. “Repetição **contada**: exatamente **12 linhas**, o total crescendo 300, 600, 900... Fim garantido pelo **range**.”
- **15:00 — Exemplo 2 (`exemplo_while.py`):** rodar; mensal 300, meta 3600. “Repetição **condicionada**: ele para **sozinho** quando bate a meta. Se eu mudar a meta para 5000, ele acha o novo número de meses” (alterar ao vivo se houver tempo).
- **16:00 — Exemplo 3 (`exemplo_erro_proposital.py`) — DEBUG AO VIVO:**
 1. Rodar; mensal 300, meta 3600.
 2. A saída dispara Mes 1 -> 0.00, Mes 2 -> 0.00, ... **sem parar**. Deixar rolar ~2 s.
 3. **Ctrl + C** para interromper. “Reparem: o total ficou **preso em 0** e o programa **nunca para**.”
 4. Pergunta à banca (avaliação em ação): “*O que está faltando dentro do laço?*” (pausa).
 5. Diagnóstico: “A condição testa **total**, mas **nada altera total** no corpo — laço infinito” (DOWNEY, 2016).
 6. **Correção ao vivo:** descomentar a linha [FIX] (`total += mensal`).
 7. Rodar de novo. “Agora **total** cresce, a condição fica falsa e ele para em 12 meses. **Corrigido**.”
- **17:40** Alt+Tab de volta aos slides.

7 Fechamento — 18:00 a 20:00 (slides 20 a 22)

Objetivo: verificar a aprendizagem e recapitular (MASETTO, 2012).

- **18:00** (verificação) — Ler a pergunta, **silêncio de ~5 s**, convidar a banca a responder. Revelar a resposta: **while True: com break no fim**, porque o corpo executa ao menos uma vez antes do primeiro teste.
- **19:00** (recap) — Percorrer os 5 checkmarks; revelar o encerramento. “Obrigado.”
- **19:40** (Referências) — Deixar à disposição; a bibliografia completa em ABNT está no material de apoio e no plano de aula.
- **20:00** Abrir para perguntas.

8 Planos de contingência

Se estiver atrasado: cortar o slide `while True + break` (o conceito já foi ao quadro); no VS Code, pular o Exemplo 2 e ir do `for` direto ao `bug`; o slide comparativo Python/C/R é explicitamente complementar.

Checklist pré-aula:

- ☐ `xaringan-themer.css` gerado (knit dos slides uma vez); `up_logo.png` e `fontawesome.html` na mesma pasta.
- ☐ Internet na projeção (ícones) ou aceitar degradação silenciosa.
- ☐ VS Code com 3 abas + terminal na pasta `codigos_py`, fonte ampliada.
- ☐ `python --version` respondendo, ou extensão Python configurada.
- ☐ Espaço no quadro para os fluxos de `for` e `while` + a nota `while True + break`; duas cores de giz/marcador.

9 Referências

ASCENCIO, A. F. G.; CAMPOS, E. A. V. **Fundamentos da programação de computadores:** algoritmos, Pascal, C/C++ (padrão ANSI) e Java. 3. ed. São Paulo: Pearson, 2012.

DOWNEY, A. B. **Pense em Python:** pense como um cientista da computação. 2. ed. São Paulo: Novatec, 2016.

MASETTO, M. T. **Competência pedagógica do professor universitário.** 2. ed. São Paulo: Summus, 2012.